# CS/CE 6378: Advanced Operating Systems
# Section 001
# Project 2

Instructor: Neeraj Mittal

Assigned on: Wednesday, October 21, 2026

This is a group project. *A group may consist of up to two students.*

*Code sharing among groups, as well as copying code from the Internet or other sources without proper authorization, is strictly prohibited and may result in disciplinary action.*

*You may use AI tools as learning aids for exploring concepts, debugging, or understanding programming techniques. However, all submitted code must reflect your own understanding and work. You must be able to explain, modify, and justify every part of your submission. If you are unable to demonstrate sufficient understanding of code included in your project, the submission may be considered a violation of academic integrity policies and may be referred to the Office of Community Standards and Conduct for review.*

*If you are uncertain whether a particular source, example, online resource, or use of AI assistance is permissible, consult the instructor before proceeding.*

You will be evaluated not only on the correctness and functionality of your project, but also on your ability to explain and defend your implementation decisions during the project demonstration. The instructor or TA may ask questions about any part of your code, design choices, algorithms, or implementation details.

## 1 Programming Language and Environment

You may implement this project in C, C++, or Java.

Because this project involves socket programming, all project demonstrations must be conducted on the csa-dcXX.utdallas.edu machines, where XX $\in$ {01, 02, ..., 45}. While you are free to develop and test your code on any platform of your choice, you are responsible for ensuring that it compiles and runs correctly on a csa-dcXX machine. Your final demonstration must be performed on a csa-dcXX machine. Demonstrations conducted on any other platform will incur a 20% penalty unless an exception has been approved in advance by the instructor.

## 2 Project Description

Implement a *mutual exclusion service* among $n$ processes using Maekawa's quorum-based distributed mutual exclusion algorithm. Your service should provide two function calls to the application: cs-enter() and cs-leave(). The first function call cs-enter() allows an application to request permission to start executing its critical section. The function call is blocking and returns only when the invoking application can execute its critical section. The second function call cs-leave()

allows an application to inform the service that it has finished executing its critical section. *The program should use preemption based strategy to avoid deadlocks.*

**Implementation Details:** Design your program so that each process or node consists of two separate modules. The top module implements the application (requests and executes critical sections). The bottom module implements the mutual exclusion service. The two modules interact using cs-enter() and cs-leave() functions.

**Application:** The application is responsible for generating critical section requests and then executing critical sections on receiving permission from the mutual exclusion service. Model your application using the following two parameters: inter-request delay, denoted by $d$, and cs-execution time, denoted by $c$. The first parameter denotes the time elapsed between when a node's current request is satisfied and when it generates the next request. The second parameter denotes the time a node spends in its critical section. Assume that both inter-request delay and cs-execution time are random variables with exponential (not uniform) probability distribution.

**Testing:** Design a mechanism to *test* the correctness of your implementation. Your testing mechanism should ascertain that at most one process is in its critical section at any time. It should be as *automated* as possible and should require minimal human intervention. *For example, visual inspection of log files to verify the correctness of the execution will not be acceptable.* You will be graded on how accurate and automated your testing mechanism is.

**Experimental Evaluation:** Evaluate the performance of your implementation with respect to message complexity, response time and system throughput using experiments for various values of system parameters, namely $n$, $d$ and $c$. Display the results of your experiments by plotting appropriate graphs. Note that each point in the graph should be obtained by averaging over multiple runs.

**Additional Work for Groups of Size Two:** Implement the second technique for avoiding deadlocks based on locking quorum-members one-by-one in a certain order.

## 3 Submission Information

To ensure that you are able to complete the project on time, we are asking you to submit your code as per the following timeline:

(i) Monday, November 2, 2026: mutual exclusion module (Maekawa's algorithm only)

(ii) Friday, November 6, 2026: mutual exclusion module (Maekawa's algorithm only) and testing mechanism

(iii) Friday, November 13, 2026: complete project (both algorithms if applicable) along with the report

All the submissions will be through eLearning. Submit all the source files necessary to compile the program and run it. Also, submit a README file that contains instructions to compile and run your program. You should also include `launcher.sh` and `cleanup.sh` scripts which run your program on the machines as configured and cleanup your processes. Finally, submit a PDF file describing the results of your experiments.

# 4 Configuration Format

Your program should run using a configuration file in the following format:

The configuration file will be a plain-text formatted file no more than 100KB in size. Only lines which begin with an unsigned integer are considered to be valid. Lines which are not valid should be ignored. The configuration file will contain $n + 1$ valid lines.

The first valid line of the configuration file contains **four** tokens. The first token is the number of nodes in the system. The second token is the mean value for inter-request delay (in milliseconds). The third token is the mean value for cs-execution time (in milliseconds). The fourth token is the number of requests each node should generate.

After the first valid line, the next $n$ lines consist of three tokens. The first token is the node ID. The second token is the host-name of the machine on which the node runs. The third token is the port on which the node listens for incoming connections.

The next $n$ lines describe the quorum of each node in the system as a list of node IDs.

Your parser should be written so as to be robust concerning leading and trailing white space or extra lines at the beginning or end of file, as well as interleaved with valid lines. The `#` character will denote a comment. On any valid line, any characters after a `#` character should be ignored.

You are responsible for ensuring that your program runs correctly when given a valid configuration file. Make no additional assumptions concerning the configuration format. If you have any questions about the configuration format, please ask the TA.

Listing 1: Example configuration file

```
9 20 10 1000

% Node information
0 dc02 1234
1 dc03 1233
2 dc06 5233
3 dc05 1232
4 dc10 1233
5 dc09 3455
6 dc15 6789
7 dc18 3467
8 dc23 2546

% Quorum definitions
0 1 2 3 6
0 1 2 4 7
0 1 2 5 8
0 3 4 5 6
1 3 4 5 7
2 3 4 5 8
0 3 6 7 8
1 4 6 7 8
2 5 6 7 8
```